

Лабораторна робота №9. Створення інтерактивного DOM та обробка подій: Event Delegation, форми та доступність динамічного інтерфейсу

Термін здачі: 16.10.2026

1. Мета роботи

Навчитися будувати інтерактивний інтерфейс на DOM API без фреймворків: створювати елементи з шаблону, обробляти події через делегування, працювати з формами й модальним діалогом і керувати фокусом так, щоб інтерфейсом можна було користуватися з клавіатури та скрінрідера. Навчитися перевіряти JavaScript-код, написаний AI-агентом, за специфікацією й поведінкою в браузері, а не за тим, що він «начебто працює».

Очікуваний результат: застосунок «Мій список», кожную частину якого спершу написав AI-агент, а ви знайшли розбіжності з ТЗ і виправили їх власноруч; «Пройдено: 33 з 33» у самоперевірці й таблиця аудиту, де кожна знахідка має правило ТЗ, доказ і коміт.

2. Цілі та результати навчання

Після виконання лабораторної роботи студент повинен:

- Знаходити елементи через `querySelector*`, розрізняти живі й статичні колекції; пам'ятати, що значення `dataset` — рядки.
- Створювати елементи з `<template>`, записувати дані через `textContent` і атрибути; пояснювати, чому `innerHTML` з даними — це XSS.
- Пояснювати фази `capturing` і `bubbling`, різницю між `target` і `currentTarget`, значення `this` в обробнику; використовувати `closest()`.
- Застосовувати `event delegation` і пояснювати, чому обробник прив'язаний до вузла, а не до селектора.
- Обробляти форми: `submit` і `preventDefault()`, надсилання клавішею Enter, `input` і `change`, `reset`, `valueAsNumber`, `Constraint Validation API`.

- Керувати фокусом: `focus()`, `tabindex`, фокус після видалення елемента, модальний `<dialog>`.
- Передавати стан допоміжним технологіям: `aria-pressed`, `aria-invalid`, `aria-describedby`, `live region (role="status")`.
- Перевіряти AI-згенерований код за ТЗ, самоперевіркою й DevTools і фіксувати аудит у Git.

3. Завдання лабораторної роботи

1. Опрацювати дослідницькі завдання (§4) і провести експерименти: для кожного **до** запуску проаналізувати код і записати очікуваний результат — власний і AI-агента.
2. Завантажити [starter-проєкт «Мій список»](#), зафіксувати його в Git без змін і переконатися, що самоперевірка показує «Пройдено: 0 з 33» (§5.1).
3. Виконати Challenges 1–5 (§5.3–5.7): для кожного AI-агент пише чернетку, ви фіксуєте її, перевіряєте за ТЗ і виправляєте власноруч.
4. Пройти ручний чек-лист ТЗ, опублікувати проєкт на GitHub Pages, оформити звіт (§6) і захистити роботу (§7).

4. Дослідницькі завдання

Перед практичною частиною опрацюйте джерела й проведіть експерименти. Загальна довідка — [Introduction to the DOM — MDN](#); вимоги доступності — [WCAG 2.2](#) і [ARIA Authoring Practices Guide](#); поведінку браузера визначає [HTML Living Standard](#).

Експерименти досліджують поведінку браузера, а не код застосунку: у них нічого не треба виправляти. Кожен експеримент описує, що є на сторінці й що робить фрагмент коду, і ставить питання для аналізу. Рядок «Де це трапиться» показує, у якій частині застосунку «Мій список» ця поведінка стане помилкою, якщо її не врахувати.

Протокол експерименту:

1. Прочитайте опис, **проаналізуйте** код і запишіть очікуваний результат — відповідь на питання «Проаналізуйте».
2. Попросіть AI-агента проаналізувати той самий код і назвати очікуваний результат. **Не показуйте** йому, що вийшло насправді, і не дозволяйте запускати код.
3. Відкрийте `experiments.html` зі starter-проєкту (§5.1) через Live Preview, оновіть сторінку й вставте фрагмент у Console цілим. Сторінку оновлюйте перед кожним експериментом: фрагменти змінюють DOM.
4. Якщо хоч один очікуваний результат не збігся з фактичним, поясніть, де саме аналіз був хибним. Відповідь на завдання з рядка «Де це трапиться» теж запишіть у

пояснення.

Результати зведіть у таблицю — вона входить у звіт:

#	Що досліджено	Мій аналіз	Аналіз AI (без запуску)	Фактичний результат	Пояснення
1	Жива і статична колекція	...	...	...	...

4.1. DOM API

Опрацювати:

- [Locating DOM elements using selectors — MDN](#)
- [HTMLCollection](#) і [NodeList](#) — MDN
- [Node: textContent](#) і [Element: innerHTML](#), розділ Security considerations — MDN
- [<template>](#), [Element: replaceChildren\(\)](#), [HTMLElement: dataset](#) — MDN

Дослідити: що повертають `querySelector` і `querySelectorAll`, якщо нічого не знайдено; живі й статичні колекції; `textContent`, `innerHTML` і `innerText`; чому вміст `<template>` інертний; що робить `cloneNode(true)`.

Експерименти:

1. **Жива і статична колекція.** У розділі 1 сторінки — шість пунктів списку з класом `item`. Фрагмент бере їх через `getElementsByClassName` і в циклі прибирає в кожного клас `item`. Останній рядок рахує, скільки елементів із цим класом лишилося.

```
const items = document.getElementsByClassName('item');
for (let i = 0; i < items.length; i++) items[i].classList.remove('item');
document.querySelectorAll('.item').length;
```

Проаналізуйте: яке число поверне останній рядок? Потім оновіть сторінку, замініть перший рядок на `const items = document.querySelectorAll('.item');` і проаналізуйте ще раз.

Де це трапиться: у будь-якому циклі, що змінює елементи колекції, поки її обходить. Поясніть, чим відрізняються ці дві колекції і яку з них безпечно обходити, коли цикл змінює DOM.

2. **Рядок із HTML чотирма способами.** `payload` створює рядок із тегом `<img>`: зображення не існує, а обробник `onerror` виводить у Console, яким способом рядок потрапив на сторінку, — але лише якщо браузер зробив із рядка справжній елемент.

Фрагмент записує рядок через `textContent`, через `innerHTML`, через `innerHTML` тимчасового `<div>`, якого немає в документі, і через `innerHTML` елемента `<template>`.

```
const payload = (via) => `
```

Проаналізуйте: які з чотирьох способів виведуть повідомлення в Console? Що буде видно в розділі 2 сторінки?

Де це трапиться: назва серіалу або нотатка, у якій є розмітка (C1, C5), і опис серіалу, що в даних TVmaze сам є HTML (C4). Поясніть, які способи безпечні для даних і чому «очистити» HTML від тегів через тимчасовий `<div>` не можна.

3. **Значення з dataset.** У розділі 3 — `<li data-id="82">`. Так само кожна картка застосунку зберігатиме `id` свого серіалу.

```
const li = document.querySelector('#exp-3 li');
[li.dataset.id, typeof li.dataset.id, li.dataset.id === 82,
Number(li.dataset.id) === 82];
```

Проаналізуйте: які чотири значення буде в масиві?

Де це трапиться: `id`, прочитаний з картки, передають у функції `src/state.js` (C2).

Поясніть, чому порівняння з числом не спрацює і як отримати число.

4. **`innerHTML +=` і обробники.** У розділі 4 — кнопка. Фрагмент додає їй обробник `click` і клікає по ній, потім дописує в розділ рядок через `innerHTML +=` і клікає знову.

```
const box = document.querySelector('#exp-4');
box.querySelector('button').addEventListener('click', () =>
console.log('клік'));
box.querySelector('button').click();
box.innerHTML += '<p>Новий рядок</p>';
box.querySelector('button').click();
```

Проаналізуйте: скільки разів Console виведе «клік»?

Де це трапиться: список, до якого нові картки «дописують» рядком HTML (C1, C2).

Поясніть, що `innerHTML +=` робить з елементами, які вже є в контейнері.

4.2. Події та event delegation

Опрацювати:

- [Introduction to events](#) і [Event bubbling](#), зокрема розділ Event delegation — MDN

- [EventTarget: addEventListener\(\)](#) — MDN, зокрема розділи про this і про повторну реєстрацію
- [Event: target](#), [Event: currentTarget](#), [Element: closest\(\)](#) — MDN
- [Button pattern — ARIA APG](#)

Дослідити: три фази поширення події; target і currentTarget; чому this у function-обробнику — елемент, а в arrow function — ні; що повертає closest(), якщо предка немає; що буде, якщо додати обробник двічі; чому делегований обробник працює для елементів, створених пізніше; що <button> уміє порівняно з <div> з обробником click; що означає isTrusted.

Експерименти:

5. **Фази поширення події.** У розділі 5 — кнопка всередині , а той — у . Фрагмент додає кожному з трьох елементів по два обробники click: один для фази capturing, другий — для bubbling. Після запуску клікніть кнопку мишею.

```
const ul = document.querySelector('#exp-5 ul');
for (const el of [ul, ul.querySelector('li'), ul.querySelector('button')])
{
  const name = el.tagName.toLowerCase();
  el.addEventListener('click', () => console.log(name, 'capture'), {
    capture: true });
  el.addEventListener('click', () => console.log(name, 'bubble'));
}
```

Проаналізуйте: у якому порядку з'являться шість рядків?

Де це трапиться: обробник на всьому списку карток (C2) отримує клік по кнопці саме завдяки цим фазам. Поясніть порядок через три фази.

6. **target, currentTarget і this.** У розділі 6 — кнопка з іконкою-зіркою (<svg>). Фрагмент додає кнопці два обробники — звичайною функцією й arrow function — і ще один

обробник на весь розділ. Після запуску клікніть мишею по зірці, потім по тексту кнопки, потім по заголовку розділу.

```
const button = document.querySelector('#exp-6 button');
button.addEventListener('click', function (event) {
  console.log('function:', event.target.tagName,
    event.currentTarget.tagName, this.tagName);
});
button.addEventListener('click', (event) => {
  console.log('arrow:', event.target.tagName, event.currentTarget ===
    button, this === window);
});
document.querySelector('#exp-6').addEventListener('click', (event) => {
  console.log('closest:', event.target.closest('button'));
});
```

Проаналізуйте: що виведе кожен рядок для кожного з трьох кліків?

Де це трапиться: кнопка «До списку» на картці теж має іконку (C2). Поясніть, від чого залежить `event.target` і що повертає `closest()`, коли кнопки серед предків немає.

7. **Той самий обробник двічі.** У розділі 7 — кнопка. Фрагмент реєструє на ній чотири обробники, що збільшують лічильник: дві однакові на вигляд arrow functions і двічі ту саму функцію `named`. Потім один раз клікає.

```
const button = document.querySelector('#exp-7 button');
let count = 0;
const named = () => count++;
button.addEventListener('click', () => count++);
button.addEventListener('click', () => count++);
button.addEventListener('click', named);
button.addEventListener('click', named);
button.click();
count;
```

Проаналізуйте: чому дорівнюватиме `count`?

Де це трапиться: обробник, який додають у функції, що перемальовує список (C2). Поясніть, коли браузер реєструє обробник повторно, а коли ні.

8. **<div> чи <button>.** У розділі 8 — `<div>`, оформлений як кнопка, і справжня `<button>`. Фрагмент додає обом обробник `click`, що виводить значення `isTrusted`. Після запуску:
- клікніть обидва елементи мишею;
 - пройдіть сторінку клавішею `Tab`;
 - коли фокус на кнопці, натисніть `Enter`, потім `Space`;

- виконайте в Console `div.click(); button.click();`.

```
const div = document.querySelector('#exp-8 .fake');
const button = document.querySelector('#exp-8 button');
div.addEventListener('click', (event) => console.log('div',
event.isTrusted));
button.addEventListener('click', (event) => console.log('button',
event.isTrusted));
```

Проаналізуйте: що виведе Console після кожної дії? Чи отримує `<div>` фокус клавішею Tab?

Де це трапиться: кнопки на картках і в діалозі (C1, C2, C4). Поясніть, що `<button>` уміє без JavaScript і що означає `isTrusted`.

4.3. Форми

Опрацювати:

- [Client-side form validation — MDN](#)
- [Constraint validation API](#) і [HTMLInputElement: setCustomValidity\(\)](#) — MDN
- [HTMLFormElement: submit event](#), [reset event](#), [requestSubmit\(\)](#) — MDN
- [Element: input event](#) і [change event](#) — MDN
- [Implicit submission — HTML Living Standard](#)

Дослідити: коли форму надсилає Enter і що станеться без `preventDefault()`; чим `submit` відрізняється від `click` по кнопці; коли настають `input` і `change`; коли настає `reset` відносно очищення полів; чим `value` відрізняється від `valueAsNumber`; що таке `validity`, `checkValidity()` і `setCustomValidity()`; що робить атрибут `novalidate`.

Експерименти:

9. **Enter у формі.** У розділі 9 — форма з полем і кнопкою «Знайти». Фрагмент додає кнопці обробник `click`, а формі — обробник `submit`. Спершу увімкніть у Console **Preserve log**, інакше записи зникнуть разом зі сторінкою. Після запуску клікніть у поле, введіть `drama` і натисніть Enter.

```
const form = document.querySelector('#exp-9 form');
form.querySelector('button').addEventListener('click', (event) =>
console.log('click', event.isTrusted));
form.addEventListener('submit', () => console.log('submit'));
```

Проаналізуйте: які записи з'являться в Console? Що станеться зі сторінкою й адресою в рядку браузера?

Де це трапиться: поле пошуку у формі фільтрів (C3) і форма нотатки (C5). Пояснить, що запускає надсилання форми і як його скасувати.

10. **value і valueAsNumber.** У розділі 10 — `<input type="number">`. Фрагмент записує в поле 7 і додає до значення 1 двома способами, потім очищує поле й повторює.

```
const input = document.querySelector('#exp-10 input');
input.value = '7';
const filled = [input.value + 1, input.valueAsNumber + 1];
input.value = '';
[filled, [input.value + 1, input.valueAsNumber + 1]];
```

Проаналізуйте: які дві пари значень поверне фрагмент?

Де це трапиться: поле «Моя оцінка» (C5), адже `saveNote()` приймає оцінку лише числом. Пояснить, якого типу кожне значення і чому для порожнього поля одне з них — NaN.

11. **setCustomValidity.** У розділі 11 — поле для оцінки від 1 до 10. Фрагмент вводить 42, позначає поле як невалідне власним повідомленням, потім виправляє значення на 5 і перевіряє валідність.

```
const input = document.querySelector('#exp-11 input');
input.value = '42';
if (input.valueAsNumber > 10) input.setCustomValidity('Не більше 10');
input.value = '5';
[input.validity.valid, input.validity.customError, input.checkValidity(),
input.validationMessage];
```

Проаналізуйте: які чотири значення буде в масиві?

Де це трапиться: перевірка оцінки у формі нотатки (C5). Пояснить, коли повідомлення, задане через `setCustomValidity`, зникає.

12. **Подія reset.** У розділі 12 — форма з полем і кнопкою «Скинути». Фрагмент додає формі обробник `reset`, що виводить значення поля, записує в поле `drama` і скидає форму.

```
const form = document.querySelector('#exp-12 form');
form.addEventListener('reset', () => console.log('y reset:',
JSON.stringify(form.elements.q.value)));
form.elements.q.value = 'drama';
form.reset();
console.log('після reset():', JSON.stringify(form.elements.q.value));
```

Проаналізуйте: які два рядки виведе Console? Потім введіть у поле щось інше й натисніть «Скинути» мишею: що виведе обробник цього разу?

Де це трапиться: кнопка «Скинути» у формі фільтрів (C3). Поясніть, коли настає подія `reset` відносно очищення полів.

4.4. Фокус, діалоги й доступність

Опрацювати:

- [HTMLElement: focus\(\)](#) і [tabindex](#) — MDN
- [<dialog>](#) і [HTMLElement: inert](#) — MDN; [Dialog \(Modal\) pattern](#) — ARIA APG
- [ARIA live regions](#), [aria-pressed](#), [aria-invalid](#), [aria-describedby](#) — MDN
- WCAG 2.2: [2.1.1 Keyboard](#), [2.4.3 Focus Order](#), [2.4.7 Focus Visible](#), [2.5.3 Label in Name](#), [3.3.1 Error Identification](#), [4.1.2 Name, Role, Value](#), [4.1.3 Status Messages](#)

Дослідити: які елементи отримують фокус без `tabindex`; `tabindex="-1"` і `tabindex="0"`; куди переходить фокус, коли елемент із фокусом видалено; що робить `showModal()` з рештою сторінки і куди повертає фокус `close()`; що таке `accessible name` і звідки його бере браузер; чому `live region` має бути в документі до появи тексту. Почути оголошення можна лише скрінрідером (Narrator, NVDA, VoiceOver) — це необов'язково.

Для експериментів 13–15 додайте в Console **Live Expression** `document.activeElement`: воно постійно показує елемент, на якому зараз фокус.

Експерименти:

13. **Видалення елемента з фокусом.** У розділі 13 — кнопка. Фрагмент ставить на неї фокус, видаляє її з документа й читає, де тепер фокус.

```
const button = document.querySelector('#exp-13 button');
button.focus();
const before = document.activeElement === button;
button.remove();
[before, document.activeElement];
```

Проаналізуйте: які два значення буде в масиві?

Де це трапиться: список перемальовано, а фокус був на кнопці старої картки (C2, C3, C5). Поясніть, куди переходить фокус і що це означає для користувача клавіатури.

14. **focus() і tabindex.** У розділі 14 — звичайний `<div>`. Фрагмент викликає на ньому

`focus()`, потім додає `tabindex="-1"` і викликає ще раз.

```
const box = document.querySelector('#exp-14 div');
box.focus();
const plain = document.activeElement === box;
box.tabIndex = -1;
box.focus();
[plain, document.activeElement === box];
```

Проаналізуйте: які два значення буде в масиві? Потім клікніть кнопку з розділу 13 і натисніть Tab: чи потрапить фокус на цей `<div>`?

Де це трапиться: коли в списку не лишилося карток, фокус треба поставити на заголовок списку (C3). Поясніть, чим `tabindex="-1"` відрізняється від `tabindex="0"`.

15. **Модальний діалог і фокус.** У розділі 15 — кнопка «Відкрити діалог», кнопка поза діалогом і `<dialog>` з однією кнопкою. Фрагмент ставить фокус на «Відкрити діалог», відкриває діалог через `showModal()`, пробує перевести фокус на кнопку поза діалогом, а потім закриває діалог.

```
const section = document.querySelector('#exp-15');
const dialog = section.querySelector('dialog');
const opener = section.querySelector('.opener');
const outside = section.querySelector('.outside');
opener.focus();
dialog.showModal();
outside.focus();
console.log('після outside.focus():',
document.activeElement.textContent.trim());
dialog.close();
console.log('після close():', document.activeElement.textContent.trim());
```

Проаналізуйте: що виведуть обидва рядки?

Потім оновіть сторінку й виконайте другий варіант: тут кнопку, з якої відкрили діалог, видалено, поки діалог відкритий.

```
const section = document.querySelector('#exp-15');
const dialog = section.querySelector('dialog');
const opener = section.querySelector('.opener');
opener.focus();
dialog.showModal();
opener.remove();
dialog.close();
```

Проаналізуйте: куди перейде фокус після `close()`? Перевірте за Live Expression.

Де це трапиться: діалог «Детальніше» (C4) і збереження нотатки, після якого картку перемальовано (C5). Поясніть, що `showModal()` робить з рештою сторінки і коли

браузер сам повертає фокус.

4.5. DOM-код, написаний AI

Дослідити, що AI-агент зазвичай робить у DOM-кодi, якщо його про це не просили:

- будує розмітку через `innerHTML` і шаблонний рядок із даними;
- додає обробник на кожну картку — один раз на старті або щоразу під час перемальовування;
- читає `event.target` без `closest()`;
- не скасовує надсилання форми;
- створює рядок статусу заново або не створює зовсім;
- вставляє опис через `innerHTML` або «очищує» його через тимчасовий `<div>`;
- робить модальне вікно з `<div>`;
- показує помилки валідації через `alert()`;
- губить фокус після перемальовування;
- додає зайвий ARIA: `role="button"` на `<div>` без підтримки клавіатури, `aria-label`, що підмінює видимий текст (WCAG 2.5.3).

Для кожного пункту запишіть, яке правило ТЗ він порушує і як його помітити.

5. Практична частина: AI-чернетка → аудит → виправлення

Starter-проект **«Мій список»** — каталог 30 серіалів [Tvmaze](#) (та сама вибірка, що в ЛР-8). У готовому застосунку користувач фільтрує серіали за назвою й жанром, додає їх до свого списку кнопкою «До списку», бачить лічильник «У списку: N», відкриває опис кнопкою «Детальніше» і зберігає свою оцінку з нотаткою. HTML, CSS, дані й модуль стану `src/state.js` уже готові. Порожній лише JavaScript, який показує картки й реагує на дії користувача, тож відкрита сторінка показує форму й порожній список.

Ваше завдання — довести застосунок до робочого стану за п'ять кроків (challenges). Кожен challenge — окрема частина застосунку, і кожен проходить той самий цикл (§5.2): AI-агент пише чернетку за коротким запитом, ви фіксуєте її в Git, перевіряєте й виправляєте власноруч. Виконуйте challenges по черзі: кожен спирається на попередні.

Як перевіряти. Точні правила для кожного challenge — у [ТЗ](#), §4. Вони пронумеровані: C4.2 — друге правило Challenge 4. Сторінка `check.html` перевіряє правила в браузері й показує кожне під тим самим номером: ✓ — виконано, X — ні, з поясненням, що саме не так. Запускайте її скільки завгодно: вона нічого не змінює в проекті. Те, чого

check.html не бачить (справжня клавіатура, панель Accessibility), перевіряє ручний чек-лист ТЗ, §6.

5.1. Підготовка

1. Завантажте [watchlist-starter.zip](#) і розпакуйте в **окремий репозиторій** або в теку lab-9/ спільного репозиторію з лабораторними. Зробіть перший коміт **без змін**.
2. Відкрийте index.html через Live Preview: форма фільтрів, порожній список, Console без помилок.
3. Відкрийте так само check.html: «Пройдено: 0 з 33». Прочитайте, чого перевірка не бачить.
4. check.html, check/, data/ і src/state.js не змінюються в жодному коміті.

5.2. Правила роботи з AI-агентом

- **Дослідження (§4).** AI аналізує код **до** запуску. У таблицю записуйте його відповідь так, як він її дав, без виправлень після запуску.
- **Challenges.** Кожен проходить чотири кроки:
 1. **Чернетка.** Дайте AI-агенту в IDE (режим, у якому він сам змінює файли проєкту) **лише** запит challenge, без ТЗ, без check.html і без власних уточнень.
 2. **Коміт без змін**, навіть якщо чернетка не працює: C2: AI draft.
 3. **Аудит.** Запустіть check.html і пункти ручного чек-листа, названі в challenge. Кожну розбіжність запишіть у таблицю аудиту (§5.8).
 4. **Виправлення власноруч.** AI може пояснити перевірку, правило чи помилку, але **не пише** код виправлення. Коміт: fix(C2): <що виправлено>, один або кілька.

Чернетка, що з першого разу пройшла все, — теж результат: запишіть в аудит, як ви її перевірили.

Приклад запиту для пояснення (режим без редагування — ask / plan):

Поясни, чому падає ця перевірка. Нижче — її назва й повідомлення та мій код. Не пиши виправлений код: назви механізм DOM або правило доступності, яке я, ймовірно, не врахував, і як перевірити це в DevTools.

У звіті вкажіть назву та версію агента, модель і дату роботи.

5.3. Challenge 1 — картки

Що має вийти. Після завантаження сторінки в списку «Серіали» — 30 карток, по одній на серіал з data/shows.json, у тому самому порядку. На картці — назва, рік, жанри,

оцінка TVmaze і дві кнопки: «До списку» та «Детальніше». Над списком рядок статусу показує, скільки серіалів знайдено. Кнопки поки нічого не роблять: це C2.

Що вже є. Шаблон картки `<template id="show-card">` в `index.html` з розміткою й місцями для даних (`data-field`), порожній список `#results` і рядок статусу `#status`. Під час запуску `main.js` викликає `renderShows()` і передає їй серіали, які треба показати.

Код: функція `renderShows(list)` у `src/render.js`.

Запит для AI-агента:

Покажи серіали з `data/shows.json` у списку `#results`: назва, рік, жанри, оцінка, кнопки «До списку» і «Детальніше».

Перевірка: рядки C1.1–C1.7 у `check.html`; ручний чек-лист, пункт 3 (імена кнопок). Знадобляться експерименти 2 і 3.

5.4. Challenge 2 — кнопки карток

Що має вийти. «До списку» додає серіал до мого списку, а повторне натискання прибирає його. Кнопка показує стан (іконка заповнюється), а лічильник «У списку: N» у `header` оновлюється. «Детальніше» відкриває опис серіалу: сам діалог буде в C4, тут достатньо викликати `openDetails(id)`. Кнопки працюють однаково мишею й клавіатурою, а також після кожного перемальовування списку.

Що вже є. `toggle(id)` і `watchlist` у `src/state.js`, лічильник `#watchlist-count` у `header`, стилі для стану [`aria-pressed="true"`], порожня поки функція `openDetails(id)` у `src/details.js`.

Код: функція `initList(refresh)` у `src/list.js`. `main.js` викликає її один раз, після першого показу списку; `refresh()` перемальовує список.

Запит для AI-агента:

Зроби, щоб «До списку» додавав серіал у мій список, а «Детальніше» відкривав опис.

Перевірка: рядки C2.1–C2.7; ручний чек-лист, пункти 1, 3 і 4. Знадобляться експерименти 4–8 і 13.

5.5. Challenge 3 — фільтри

Що має вийти. Форма над списком фільтрує картки: пошук за частиною назви спрацьовує під час введення, жанр і «Лише мій список» — одразу після вибору, і фільтри діють разом. Рядок статусу щоразу показує, скільки знайдено. Enter у полі пошуку не перезавантажує сторінку, а «Скинути» повертає всі 30 серіалів. Якщо в режимі «Лише мій

список» прибрати серіал зі списку, його картка зникає, а фокус переходить на сусідню картку.

Що вже є. Форма `#filters` з полями `query`, `genre`, `mine` і кнопкою «Скинути». Функція `visibleShows({ query, genre, mine })` у `src/state.js` уже фільтрує серіали: їй треба передати значення форми й перемалювати список.

Код: функції `readFilters()` і `initFilters(refresh)` у `src/filters.js`; фокус після прибирання картки — у `src/list.js`.

Запит для AI-агента:

Додай фільтрацію за назвою і жанром і перемикач «лише мій список».

Перевірка: рядки C3.1–C3.7; ручний чек-лист, пункти 1, 2 і 5. Знадобляться експерименти 9, 12, 13 і 14.

5.6. Challenge 4 — діалог «Детальніше»

Що має вийти. «Детальніше» відкриває модальне вікно з назвою серіалу, його описом і посиланням на сторінку TVmaze. Поки вікно відкрите, решта сторінки недоступна. Esc або «Скасувати» закривають його, і фокус повертається на кнопку, з якої вікно відкрили. Опис у даних — HTML-розмітка TVmaze, а показати його треба як звичайний текст.

Що вже є. В `index.html` є `<dialog id="details">` із заголовком `#details-title`, місцем для опису `#details-summary`, посиланням `#details-link` і формою нотатки (вона знадобиться в C5). JavaScript лише заповнює й відкриває діалог. Дані серіалу повертає `getShow(id)` з `src/state.js`.

Код: функції `openDetails(id)` і `initDetails(refresh)` у `src/details.js`. `openDetails` викликає кнопка «Детальніше» (C2), а `initDetails` підключає закриття діалогу.

Запит для AI-агента:

Зроби діалог «Детальніше» з описом серіалу.

Перевірка: рядки C4.1–C4.5; ручний чек-лист, пункти 1–3. Знадобляться експерименти 2 і 15.

5.7. Challenge 5 — моя оцінка й нотатка

Що має вийти. У діалозі під описом — форма: «Моя оцінка» (обов'язкове ціле число від 1 до 10) і «Нотатка» (необов'язкова). Неправильної оцінки форма не приймає: під полем з'являється текст помилки, а діалог лишається відкритим. Після збереження діалог закривається, картка показує «Моя оцінка: 8/10» і нотатку, рядок статусу повідомляє про

збереження, а фокус повертається на «Детальніше» цієї картки. Коли діалог відкривають знову, форма показує збережені дані саме цього серіалу.

Що вже є. Форма `#note-form` з полями `rating` і `note`, місцем для тексту помилки `#note-rating-error` і кнопками «Зберегти» та «Скасувати»; `saveNote()` і `notes` у `src/state.js`; місця для моєї оцінки й нотатки в шаблоні картки.

Код: функція `initDetails(refresh)` у `src/details.js` (заповнювати форму під час відкриття — в `openDetails`), показ оцінки на картці — у `src/render.js`.

Запит для AI-агента:

Додай у діалог форму «моя оцінка й нотатка» з валідацією.

Перевірка: рядки C5.1–C5.7; ручний чек-лист, пункти 1, 2, 3 і 5. Знадобляться експерименти 2, 9, 10, 11 і 15.

5.8. Аудит і коміти

Аудит — таблиця, щонайменше один рядок на challenge, і вона входить у звіт:

Challenge	Правило ТЗ	Що було в AI-чернетці	Доказ (check / DevTools / клавіатура)	Виправлення (коміт)
C2	C2.4	<code>event.target.dataset.action</code> без <code>closest()</code>	X C2.4: клік по іконці отримав <code><svg></code>	<code>fix(C2): find the button with closest()</code>

Коміти кожного challenge — `C<N>: AI draft`, потім `fix(C<N>): ...`, тож `git diff` між ними і є аудитом. Після C5 пройдіть ручний чек-лист ТЗ і опублікуйте проєкт на GitHub Pages (інструкція — у `README.md`).

6. Вимоги до звіту

Тема листа:

[Веб] ЛР-9: Створення інтерактивного DOM та обробка подій: Event Delegation, форми та доступність динамічного інтерфейсу

У листі вкажіть:

GitHub repository: `https://github.com/<username>/<repository>`
GitHub Pages: `https://<username>.github.io/<repository>/`

За адресою Pages викладач відкриє і застосунок, і `check.html`.

Звіт повинен містити:

1. таблицю експериментів §4 (усі 15) і короткі висновки до кожного підрозділу;
2. назву та версію AI-агента, модель, дату роботи;
3. таблицю аудиту (§5.8) для всіх п'яти challenges;
4. результат `check.html` на опублікованій сторінці — «Пройдено: 33 з 33»;
5. ручний чек-лист ТЗ з результатом кожного пункту;
6. підсумок: що AI зробив правильно без підказки і чого не зробив жодного разу, доки його не попросили.

7. Усний захист

Формат здачі передбачає усну співбесіду за результатами практичної реалізації. Виправлення пишете ви, тож на захисті змінюєте код наживо, без AI, і відповідаєте на питання з демонстрацією екрана.

Орієнтовні питання:

Аудит

- Покажіть одну знахідку в AI-чернетці: перевірку, що її виявила, причину й коміт, який її виправив.
- Змініть код наживо: кнопка «Прибрати нотатку» на картці через наявний делегований обробник, фокус на тексті нотатки після збереження або `<select>` сортування у формі фільтрів.

Події

- Чому клік по іконці має інший `target`? Що повертає `closest()` для кліку поза кнопками?
- Чому картка, створена після запуску, працює з делегуванням, але не з обробниками на кожній картці?

Форми й фокус

- Чому Enter у полі пошуку перезавантажує сторінку, якщо `submit` ніхто не обробляє?
- Куди переходить фокус, коли елемент із фокусом видалено? Що у вашому коді цьому запобігає?

DOM і безпека

- Чому для нотатки достатньо `textContent`? Що не так із `div.innerHTML = summary;` `div.textContent`?

AI

- Чого AI не зробив жодного разу, доки його не попросили? Чому, на вашу думку?

8. Оцінювання

1. Успішний захист — **2 бали**.
2. Додатковий **1 бал** за однієї з умов:
 - термін здачі до **16.10.2026**, АБО
 - робота на парі.